

Esame OS Internals (Cabodi)

Data: 16/01/2023

Nota Generale: SE I RISULTATI SONO NUMERI, RIPORTARE PASSAGGI INTERMEDI RILEVANTI E/O FORMULE USATE. LE RISPOSTE SI/NO VANNO MOTIVATE.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB, Cache e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer), su cui si misura sperimentalmente un "hit ratio" del 90%. La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti: $p1$, $p2$ e d , rispettivamente di 40 bit, 12 bit e 12 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

Si risponda alle seguenti domande:

- A) Supponendo che non sia presente una memoria cache, e che la memoria RAM abbia tempo di accesso di 200 ns, si calcoli il tempo effettivo di accesso (EAT) per il caso proposto (TLB hit ratio = 90%), assumendo che il tempo di accesso alla TLB sia trascurabile.
- A) Si vuole ora tener conto del fatto che il sistema dispone di memoria cache, per la quale si sa che la probabilità di (cache) miss è del 10% (un accesso non in cache ogni 10) e che la cache ha tempo di accesso 30 ns. Si ricalcoli EAT, con gli stessi dati per quanto riguarda la TLB e ipotizzando, nel caso di cache miss, un tempo di accesso alla RAM di 230 ns.
- A) Si consideri ora la frequenza di page fault p . Si faccia riferimento al caso della domanda B (presenza di cache) e si ipotizzi che un page fault sia servito in 5 ms. Quanto deve valere p affinché si possa garantire un degrado massimo del 25% per EAT (causato sia dalla TLB che dai page fault)? Si tratta di un valore massimo o minimo per p ?

Domanda 2 (File System, Linked List e FAT)

Contesto: Sia dato un file di dimensione 160MB, in un file system di dimensione complessiva 400GB. Si vogliono confrontare le possibili organizzazioni di tale file, su file system con allocazione non contigua, secondo gli schemi:

- "linked list allocation"
- "File Allocation Table (FAT)"

Si supponga che i blocchi su disco (sia per i dati che per i metadati) abbiano dimensione 8KB, e che i puntatori e gli indici abbiano dimensione 32 bit.

Si dica, per ognuna delle 2 soluzioni:

- A) Quanti blocchi (di dato e/o di metadato) sono necessari per il file (attenzione: Si richiede il conteggio ESATTO, eventualmente espresso in termini di potenze di 2, ricordando che $1K = 2^{10}$ e $1M = 2^{20}$)? Nel caso della FAT, si dica quale percentuale di FAT è utilizzata per il file.
- A) Quante letture in RAM e quanti accessi a disco (per trasferire un blocco in RAM) sono necessari per leggere il byte n. 42070 all'interno del file? Si consideri un accesso diretto al file. Si supponga di non utilizzare buffer cache, e che un puntatore o indice venga letto in RAM con una sola lettura. Si supponga poi che il File Control Block (FCB) sia già in memoria RAM, che ogni accesso a disco legga o scriva un blocco di 8KB, e che la FAT (qualora utilizzata) sia già in RAM.
-

Domanda 3 (Dimensione Pagina e Prestazioni)

Contesto: Si consideri un sistema con paginazione. Si dica, per ognuna delle caratteristiche elencate, se sia preferibile una dimensione di pagina MAGGIORE o MINORE, motivando la risposta.

- Frammentazione (interna)
- Dimensione della Page Table (PT)
- Sovraccarico I/O (I/O overhead)
- Numero di errori di pagina (Page Faults)
- Località
- Dimensioni/copertura TLB

2 SECTION 2: OS161

Domanda 4 (Creazione Thread in OS161)

Contesto: Si fornisce una rappresentazione (parziale) della funzione OS161 `thread_fork`.

```
int thread_fork(const char *name, struct proc *proc,
               void (*entrypoint)(void *data1, unsigned long data2),
               void *data1, unsigned long data2) {
    struct thread *newthread;
    int result;
    newthread = thread_create(name);

    /* Allocate a stack */
    newthread->t_stack = kmalloc(STACK_SIZE);

    /* Thread subsystem fields */
    newthread->t_cpu = curthread->t_cpu;

    /* Attach the new thread to its process */
    result = proc_addthread(proc, newthread);

    /* Set up the switchframe so entrypoint() gets called */
    switchframe_init(newthread, entrypoint, data1, data2);

    /* Lock the current cpu's run queue and make the new thread runnable */
    thread_make_runnable(newthread, false);

    return 0;
}
```

Basandosi sul codice riportato e su quanto appreso in lezioni e laboratori, si risponda alle seguenti domande:

- B) Che cos'è `entrypoint` e a cosa serve nel contesto della creazione di un thread?
- B) Lo stack allocato (di dimensione `STACK_SIZE`) è stack kernel oppure user? (Motivare).
- B) Che cosa fa la funzione `thread_make_runnable`? Manda in esecuzione il thread? (Se no, motivare; se sì, quando?).
- B) E' possibile che l'esecuzione arrivi all'istruzione finale (`return 0;`), oppure, una volta entrati in `thread_make_runnable`, non è possibile che il controllo sia ritornato alla `thread_fork`?

Domanda 5 (Gestione Memoria `dumbvm` e Page Table custom)

Contesto: Si consideri, in OS161, la realizzazione della gestione della memoria virtuale. Si risponda alle seguenti domande:

- B) Si supponga di voler realizzare in `dumbvm` la gestione dei frame liberi mediante free-list, invece che con bitmap (come fatto nella soluzione proposta per il lab 2).

- A1)** Si dica se la free list può essere una lista non ordinata oppure se deve essere ordinata. (Motivare).
- A2)** Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici oppure logici? (Motivare).
- B)** Si supponga di voler invece realizzare una PAGE TABLE a livello di singolo processo. Viste le dimensioni ridotte della RAM fisica, si ipotizzi che una casella (entry) della PT occupi solo 2 Byte. Si Ipotizzi di NON voler gestire un heap per il processo: il processo contiene quindi unicamente (come fatto in `dumbvm`) i segmenti di codice, di dati e lo stack. Si sa che:
- La costante `PAGE_SIZE` è definita come 4096.
 - Il segmento 1 (codice) inizia all’indirizzo logico `0x400000` e ha dimensione 10112 Byte.
 - Il segmento 2 (dati) inizia all’indirizzo logico `0x412000` e ha dimensione 8028 Byte.
 - Lo stack ha dimensione 18 pagine, collocate ai massimi indirizzi logici possibili.
- B1)** Si calcoli la dimensione complessiva dell’address space e, al suo interno, il numero di pagine effettivamente usate (valide).
- B2)** Si calcoli quale sarebbe la dimensione di una page table standard.
- B3)** Si calcoli quale sarebbe la dimensione di una page table gerarchica a due livelli, supponendo di utilizzare 11 bit per p_2 : si ricorda che l’indirizzo logico è (p_1, p_2, d) .